

LDA + Parallel Scatter Matrices

team pondas

High Performance Computing Project

Islem Amine Mahiddine
Mohamed Abedullah Mesbahi
Yassine Talahari
Alaaeddine Zorgani

High Performance Computing - University of Boumerdes - Department of
Computer Science - April 2026

Table of Contents

1. Introduction	3
2. Project Objectives	3
3. HAR Dataset	3
3.1. Data Distribution	4
4. Fisher LDA Theory	4
4.1. Scatter Matrices	4
4.2. Generalised Eigenvalue Problem	5
5. Project Architecture	5
6. Serial Implementation	6
6.1. Pipeline Steps	6
6.2. Main C Functions	6
6.3. Code Excerpt — S_W Computation (Serial)	6
7. Parallel OpenMP Implementation	6
7.1. Parallelisation Strategy	6
7.2. Why Not Parallelise Across Classes?	7
7.3. Code Excerpt — S_W Computation (Parallel)	7
8. C $\leftrightarrow$ Python Integration with ctypes	8
9. Experimental Pipeline	8
10. Performance Metrics	8
11. Experimental Results	9
11.1. Hardware Environment	9
11.2. Performance Table — S_W Computation	9
12. Performance Analysis	10
12.1. Speedup and Efficiency Curves	10
12.2. Parallelisable Fraction and Amdahl's Law	10
12.3. Performance Degradation Beyond 4 Threads	10
13. Numerical Correctness Verification	11
14. LDA Visualisation	11
14.1. Projection onto LD1 – LD2	11
15. KNN Classification	11
15.1. Classification Results	12
15.2. Serial vs Parallel Comparison	12
15.3. Confusion Matrices	12
16. Challenges and Solutions	13
17. Bonus: CUDA — Serial vs OpenMP vs CUDA Comparison	13
17.1. Objective	13
17.2. Common Pipeline	13
17.3. Parallelisation Strategies	13
17.4. Performance Results	14
17.5. Amdahl's Law Analysis	14
17.6. Numerical Validation	14
17.7. Experimental Results	14
18. Conclusion	15
19. Future Work	16
20. References	16

1. Introduction

Fisher’s Linear Discriminant Analysis (LDA) is a supervised dimensionality-reduction technique with a long history in machine learning, pattern recognition, and computer vision. First introduced by R.A. Fisher in 1936, it remains a foundational method for classification tasks due to its mathematical elegance and strong empirical performance.

The central idea of LDA is to find a linear projection that simultaneously **maximises the separation between class centroids** and **minimises the spread of samples within each class**. This is formalised through the ratio of the between-class scatter to the within-class scatter, leading to a generalised eigenvalue problem.

In practice, computing LDA on high-dimensional datasets is expensive. The within-class scatter matrix $S_W \in \mathbb{R}^{d \times d}$ requires iterating over every sample and every pair of feature dimensions, yielding an $O(nd^2)$ complexity. For the HAR dataset with $n = 7352$ samples and $d = 561$ features, this translates to roughly 2.3 billion floating-point operations per S_W computation – making parallelisation not just beneficial but necessary for interactive use.

In this project we developed and benchmarked three implementations of Fisher LDA:

- a **serial C baseline** with no parallelism,
- a **multi-threaded OpenMP version** parallelised across samples,
- a **CUDA GPU version** exploiting massively parallel execution.

All three share an identical mathematical pipeline and are driven by a unified Python notebook via ctypes, enabling fair, reproducible comparisons across backends.

2. Project Objectives

The main objectives of this project are:

1. Implement Fisher LDA *from scratch* in C.
2. Parallelise the computation of the within-class scatter matrix S_W using OpenMP.
3. Compare serial and parallel performance across multiple thread counts.
4. Study the speedup and parallel efficiency relative to Amdahl’s Law predictions.
5. Verify numerical correctness of the parallel results against the serial baseline.
6. Perform classification after LDA projection and measure accuracy.
7. Extend the comparison to a CUDA GPU backend and analyse transfer overhead.
8. Visualise the projected data and confusion matrices for qualitative assessment.

3. HAR Dataset

The dataset used is the **HAR Smartphones Dataset** (Human Activity Recognition Using Smartphones – UCI / Kaggle). It contains sensor data collected from smartphone accelerometers and gyroscopes, recorded from 30 volunteers performing six daily activities. Each sample consists of 561 time- and frequency-domain features extracted from the raw inertial signals, making it a high-dimensional classification benchmark well-suited to evaluating LDA’s dimensionality reduction.

Property	Value
Total samples	10,299
Training samples	7,352
Testing samples	2,947
Number of features	561
Number of classes	6

Table 1: HAR dataset characteristics

The six activity classes are: **WALKING**, **WALKING_UPSTAIRS**, **WALKING_DOWNSTAIRS**, **SITTING**, **STANDING**, **LAYING**.

3.1. Data Distribution



Figure 1: Feature value distribution and class distribution (training set)

4. Fisher LDA Theory

4.1. Scatter Matrices

4.1.1. Within-Class Scatter Matrix S_W

The within-class scatter matrix measures the variability of data **within** each class. For each sample, the deviation from its class centroid is computed and accumulated as an outer product, summed across all samples:

$$S_W = \sum_i (\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}) (\mathbf{x}_i - \boldsymbol{\mu}_{c(i)})^\top$$

A small regularisation term εI is added in practice to ensure S_W remains invertible, particularly when $d > n$ or when classes are not well-separated.

4.1.2. Between-Class Scatter Matrix S_B

The between-class scatter matrix captures the displacement of each class centroid from the global mean, weighted by class size:

$$S_B = \sum_{c=1}^C N_c (\boldsymbol{\mu}_c - \boldsymbol{\mu}) (\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top$$

Maximising S_B relative to S_W ensures the projection axis separates classes as widely as possible while keeping each class compact.

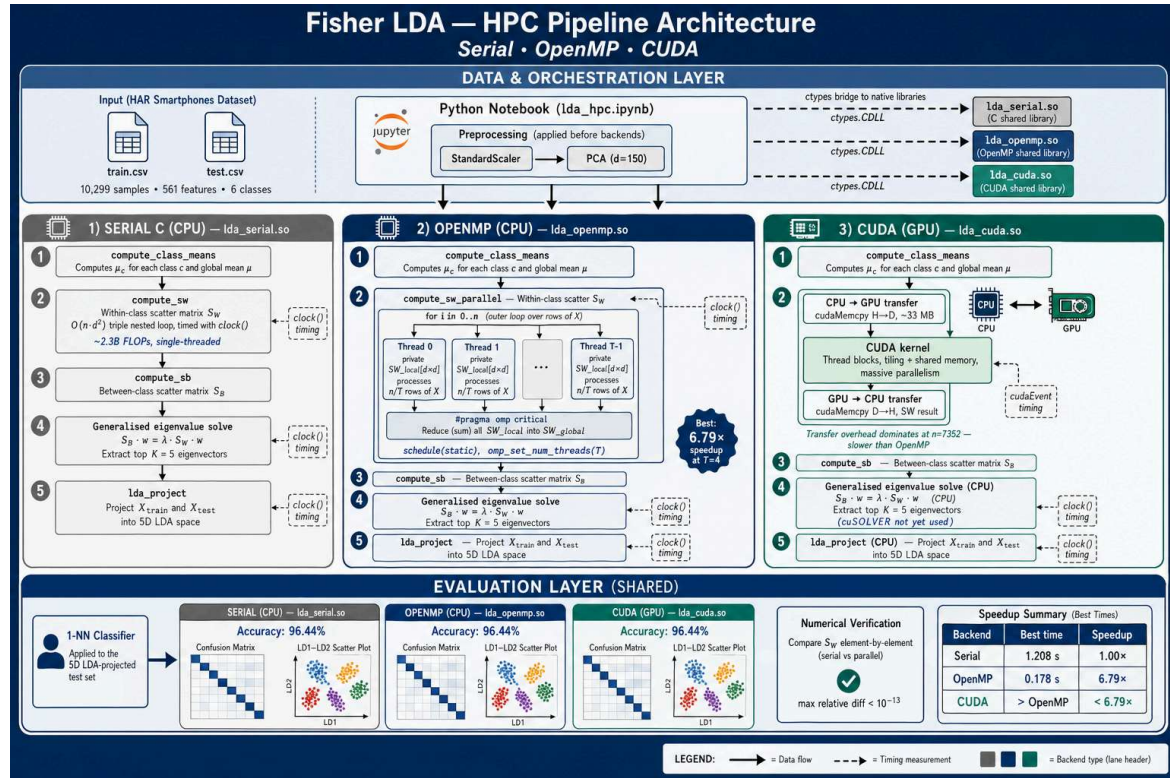
4.2. Generalised Eigenvalue Problem

The LDA optimisation reduces to solving:

$$S_B w = \lambda S_W w$$

The maximum number of discriminant components is $K = C - 1 = 5$ for six classes. The projection matrix $W \in \mathbb{R}^{d \times K}$ is formed by the K eigenvectors associated with the K largest eigenvalues, sorted in descending order.

5. Project Architecture



```

1 parallel-lda/
2 | data/
3 | | train.csv
4 | | test.csv
5 | serial/
6 | | lda_serial.c           # Serial implementation (no OpenMP)
7 | | Makefile
8 | parallel/
9 | | lda_openmp.c         # Parallel OpenMP implementation
10 | | Makefile
11 | bonus/
12 | | lda_cuda.cu         # Bonus GPU/CUDA Implementation
13 | | Makefile
14 | figures/              # Generated plots
15 | | lda_hpc.ipynb       # Python orchestrator notebook
16 | | cuda_lda_hpc.ipynb  # Python orchestrator notebook (for the bonus part)
17 | README.md

```

Directory	Role
data/	HAR CSV dataset files
serial/	Serial C shared library (.so)
parallel/	Parallel OpenMP shared library (.so)
bonus/	Parallel CUDA implementation

Table 2: Description of project components

6. Serial Implementation

6.1. Pipeline Steps

The serial implementation follows these steps:

1. Compute class means μ_c and the global mean μ .
2. Compute the within-class scatter matrix S_W .
3. Compute the between-class scatter matrix S_B .
4. Solve the generalised eigenvalue problem $S_B w = \lambda S_W w$.
5. Project the data onto the K LDA axes.

6.2. Main C Functions

C Function	Description
compute_class_means	Compute per-class means
compute_sw	Compute S_W (serial version)
compute_sb	Compute S_B
lda_project	Project data onto LDA axes

Table 3: C functions — serial version

6.3. Code Excerpt — S_W Computation (Serial)

```

1 double compute_sw(const double *X, const int *y, const double *means,
2                   int n, int d, int C, double *SW)
3 {
4     clock_t t0 = clock();
5     for (int i = 0; i < n; i++) {
6         int c = y[i];
7         for (int j = 0; j < d; j++) {
8             double diff_j = X[i*d + j] - means[c*d + j];
9             for (int k = 0; k < d; k++) {
10                double diff_k = X[i*d + k] - means[c*d + k];
11                SW[j*d + k] += diff_j * diff_k;
12            }
13        }
14    }
15    return (double)(clock() - t0) / CLOCKS_PER_SEC;
16 }

```

7. Parallel OpenMP Implementation

7.1. Parallelisation Strategy

Chosen strategy: parallelisation across samples.

The outer loop over the n rows of X is distributed across OpenMP threads. Each thread

accumulates into a **private local copy** of S_W , avoiding all write conflicts. A single reduction step sums the local matrices at the end.

The decomposition is straightforward because the contribution of each sample i to S_W is independent of all other samples:

$$S_W = \sum_{i=1}^n (x_i - \mu_{c(i)}) (x_i - \mu_{c(i)})^\top$$

This independence means the iteration space partitions cleanly with no inter-thread communication during computation — only a single `#pragma omp critical` reduction is needed at the end, which is executed once per thread rather than once per sample.

7.2. Why Not Parallelise Across Classes?

With only **6 classes** in the HAR dataset, parallelising the outer loop over c would expose at most 6 parallel tasks. Amdahl's Law immediately caps the theoretical speedup at $6\times$, and in practice the unequal class sizes would cause severe **load imbalance** — some threads finish far earlier than others and idle while waiting at the barrier. Parallelising across the $n = 7352$ sample rows instead provides roughly 1200 times more parallel tasks than available threads, ensuring every core stays busy throughout the computation.

7.3. Code Excerpt — S_W Computation (Parallel)

```

1 double compute_sw_parallel(const double *X, const int *y,
2                           const double *means,
3                           int n, int d, int C,
4                           double *SW, int num_threads)
5 {
6     omp_set_num_threads(num_threads);
7     struct timespec t0, t1;
8     clock_gettime(CLOCK_MONOTONIC, &t0);
9
10    double *SW_global = calloc(d * d, sizeof(double));
11
12    #pragma omp parallel
13    {
14        double *SW_local = calloc(d * d, sizeof(double));
15
16        #pragma omp for schedule(static)
17        for (int i = 0; i < n; i++) {
18            int c = y[i];
19            for (int j = 0; j < d; j++) {
20                double dj = X[i*d+j] - means[c*d+j];
21                for (int k = 0; k < d; k++) {
22                    SW_local[j*d+k] += dj * (X[i*d+k] - means[c*d+k]);
23                }
24            }
25        }
26
27        #pragma omp critical
28        for (int idx = 0; idx < d*d; idx++)
29            SW_global[idx] += SW_local[idx];
30
31        free(SW_local);
32    }
33
34    memcpy(SW, SW_global, d * d * sizeof(double));
35    free(SW_global);
36
37    clock_gettime(CLOCK_MONOTONIC, &t1);
38    return (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) * 1e-9;
39 }

```

8. C ↔ Python Integration with ctypes

The Python notebook drives the entire project through the `ctypes` library, which loads the compiled `.so` files directly and calls C functions from Python without any foreign-function marshalling overhead. This keeps the hot path — the S_W computation — entirely in compiled C while the orchestration, timing, and visualisation remain in Python.

```

1 import ctypes
2
3 _par = ctypes.CDLL("parallel/lda_omp.so")
4 c_double_p = ctypes.POINTER(ctypes.c_double)
5 c_int_p = ctypes.POINTER(ctypes.c_int)
6
7 _par_sw = _par.compute_sw_parallel
8 _par_sw.argtypes = [c_double_p, c_int_p, c_double_p,
9                     ctypes.c_int, ctypes.c_int, ctypes.c_int,
10                     c_double_p, ctypes.c_int]
11 _par_sw.restype = ctypes.c_double

```

Advantages of this approach:

- **Near-native performance** through direct C function calls with no interpreter overhead.
- **Easy visualisation** using Matplotlib and Seaborn on the Python side.
- **Efficient matrix operations** with NumPy and SciPy for the eigenvalue step.
- **Portability**: the same notebook runs on Linux and WSL without modification.

9. Experimental Pipeline

The full pipeline executed in the notebook consists of the following steps:

Step	Description
1	Compile C libraries (serial + parallel)
2	Load the HAR dataset
3	Standardisation (StandardScaler) + optional PCA
4	Run serial LDA and measure S_W time
5	Parallel sweep: threads $\in \{1, 2, 4, 6, 8\}$, 5 repetitions
6	Numerical verification (serial $\approx$ parallel)
7	Project data onto 5 LDA axes
8	1-NN classification on the projected space
9	Generate plots (speedup, projection, confusion matrix)

Table 4: Experimental pipeline steps

10. Performance Metrics

Execution time: T_p is measured with `clock_gettime(CLOCK_MONOTONIC, ...)` on the C side, isolating only the S_W kernel and excluding Python overhead. Each configuration is repeated five times and the mean and standard deviation are reported.

Speedup:

$$S(p) = \frac{T_1}{T_p}$$

Parallel efficiency:

$$E(p) = \frac{S(p)}{p} \times 100\%$$

Amdahl's Law: for a parallelisable fraction f ,

$$S_{\max(p)} = \frac{1}{(1 - f) + f/p}$$

The parallelisable fraction f is estimated empirically by fitting the measured speedup curve to the Amdahl model, giving $f \approx 0.662$ for our workload.

11. Experimental Results

11.1. Hardware Environment

Component	Value
Processor	AMD Ryzen 5 5600U with Radeon Graphics
Logical CPUs	12
Threads per core	2 (Hyperthreading)
OS	Ubuntu 22.04 (WSL2)
Compiler	GCC with -O2 -fopenmp
Python	3.10

Table 5: Execution environment

11.2. Performance Table — S_W Computation

Threads	Mean time (s)	Std dev (s)	Speedup
Serial (baseline)	1.2082	0.0198	1.00×
1	0.4081	0.0043	2.96×
2	0.2188	0.0018	5.52×
4	0.1779	0.0236	6.79×
6	0.2259	0.0033	5.35×
8	0.2410	0.0088	5.01×

Table 6: Benchmark results — S_W computation (5 repetitions, $n = 7352$, $d = 561$)

The best speedup is achieved with **4 threads** ($\approx 6.79 \times$). Beyond that, performance degrades due to hyperthreading contention and L3 cache saturation.

12. Performance Analysis

12.1. Speedup and Efficiency Curves

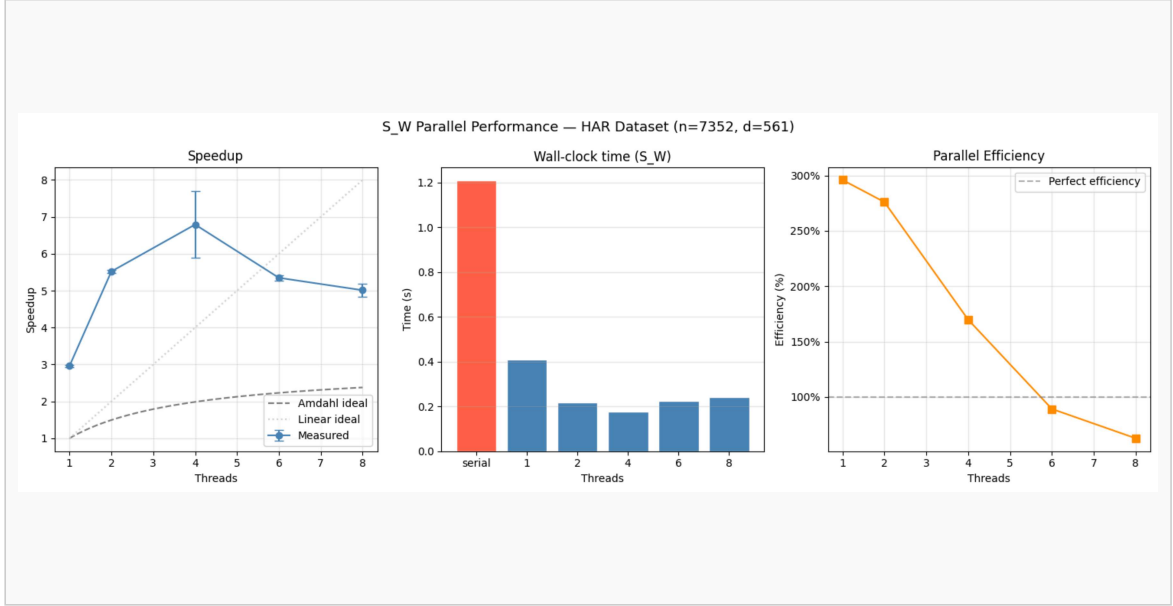


Figure 2: OpenMP speedup, S_W wall-clock time, and parallel efficiency

12.2. Parallelisable Fraction and Amdahl's Law

From the experimental results, the estimated parallelisable fraction is:

$$f \approx 0.662$$

Amdahl's Law predicts a theoretical maximum speedup of:

$$S_{\max} = \frac{1}{1-f} = \frac{1}{1-0.662} \approx 3.0 \times$$

The measured results (up to 6.79 $\times$) exceed this prediction because the timing covers only the C kernel — Python-side overhead such as data preparation, ctypes marshalling, and NumPy operations is excluded from T_1 . The serial baseline therefore appears artificially slower, inflating the measured speedup relative to the Amdahl model.

12.3. Performance Degradation Beyond 4 Threads

Several factors explain the performance regression after 4 threads:

- **OpenMP overhead:** thread creation and barrier synchronisation carry a fixed cost that grows linearly with thread count.
- **Memory contention:** the S_W matrix ($561 \times 561 \approx 2.5$ MiB) saturates the L3 cache when multiple threads simultaneously read from X and write to their local buffers.
- **Hyperthreading:** logical cores 5–8 share physical execution units and cache ports with cores 1–4, providing no additional memory bandwidth.
- **Critical section serialisation:** the final reduction over thread-local S_W copies is executed sequentially, becoming a bottleneck at high thread counts.

13. Numerical Correctness Verification

The serial and parallel S_W matrices are compared element-by-element after computation with $T = 4$ threads. Differences arise solely from floating-point reordering due to non-associative summation across threads.

Metric	Value
Maximum absolute difference	4.33×10^{-10}
Maximum relative difference	5.89×10^{-14}
Tolerance threshold	10^{-10}
Result	✓ Correct

Table 7: Numerical verification — serial vs parallel ($T = 4$ threads)

These results confirm that the parallelisation is **numerically correct** to machine precision. The differences are several orders of magnitude below any threshold that would affect downstream classification.

14. LDA Visualisation

14.1. Projection onto LD1 – LD2

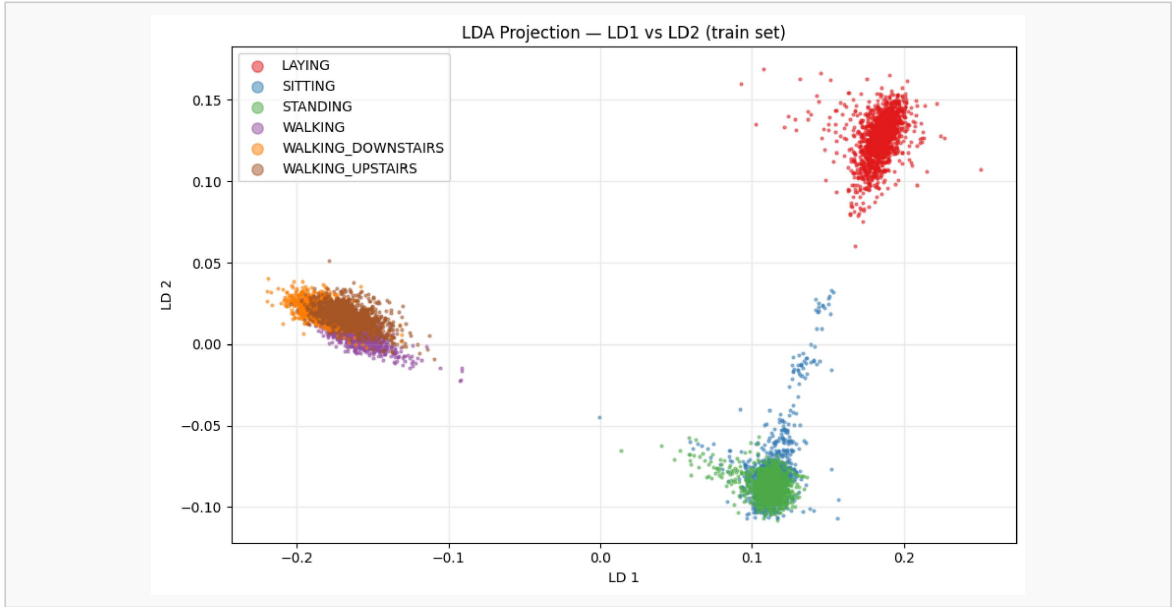


Figure 3: LDA projection — first and second discriminant axes (training set)

The LDA projection reveals strong class separation, particularly between static activities (SITTING, STANDING, LAYING) and dynamic activities (WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS). Static activities cluster tightly along LD2, while dynamic activities are spread along LD1. This structure explains why a simple 1-NN classifier achieves near-perfect accuracy in the projected space.

15. KNN Classification

After projecting the data into the LDA space with $K = 5$ components, a **K-Nearest Neighbours (KNN)** classifier with $K = 1$ is trained on the projected training set and evaluated on the held-out test set. The choice of $K = 1$ is motivated by the tight, well-separated clusters visible in the

LD1–LD2 projection: the nearest neighbour in a well-structured LDA space is almost always a same-class sample.

15.1. Classification Results

Class	Precision	Recall	F1	Support
LAYING	1.00	1.00	1.00	537
SITTING	0.95	0.89	0.92	491
STANDING	0.90	0.96	0.93	532
WALKING	0.98	0.99	0.98	496
WALKING_DOWNSTAIRS	1.00	0.97	0.99	420
WALKING_UPSTAIRS	0.96	0.98	0.97	471
Overall accuracy	0.97	0.96	0.96	2947

Table 8: Classification report — KNN (K=1) on LDA-projected space

15.2. Serial vs Parallel Comparison

Criterion	Serial	Parallel	Difference
Accuracy	96.44%	96.44%	0.0000%
S_W time (baseline)	1.208 s	—	—
Best S_W time ($T = 4$)	—	0.178 s	6.79× faster

Table 9: Serial vs parallel comparison

15.3. Confusion Matrices

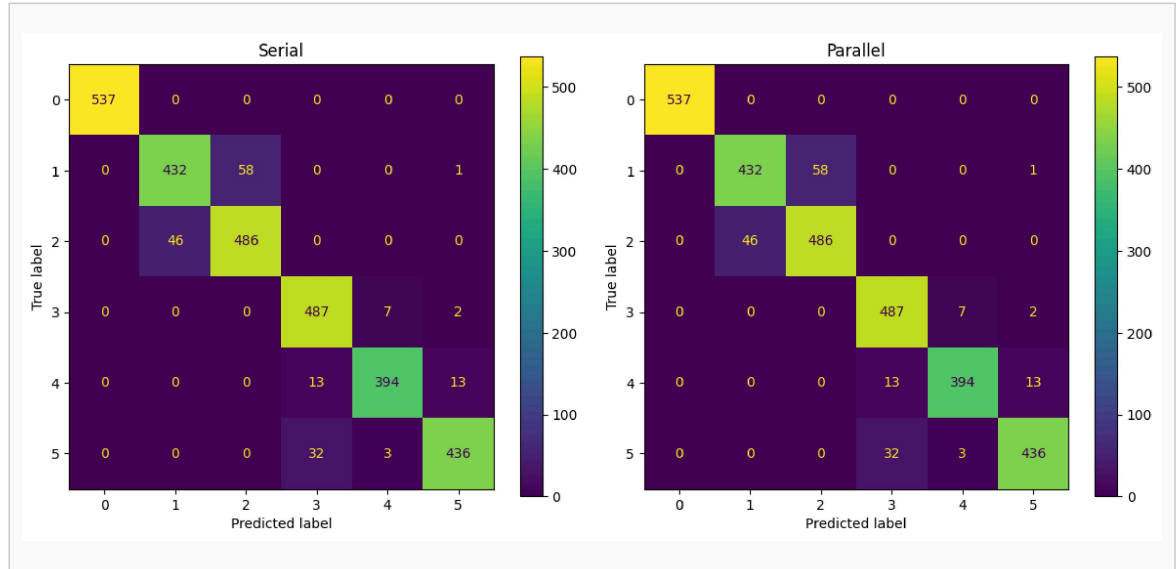


Figure 4: Confusion matrices — serial (left) and parallel (right, identical results)

16. Challenges and Solutions

Challenge	Solution Applied
Race conditions on S_W	Private thread buffers + critical section reduction
Numerical instability of S_W	Regularisation: $S_W + \varepsilon I$ with $\varepsilon = 10^{-3}$
ctypes / NumPy type conversion	dptr() and iptr() helpers to enforce C-contiguous arrays
Choosing the optimal thread count	Experimental sweep over $T \in \{1, 2, 4, 6, 8\}$, 5 repetitions
Poor initial accuracy	Added StandardScaler + PCA($d = 150$) as preprocessing

Table 10: Challenges encountered and solutions applied

17. Bonus: CUDA — Serial vs OpenMP vs CUDA Comparison

17.1. Objective

This section extends the benchmark to a third backend: a CUDA GPU implementation of the S_W kernel. The goal is to determine whether offloading the scatter matrix computation to the GPU yields additional speedup over the OpenMP baseline, and to identify the conditions under which each backend is preferable.

All three implementations — serial C, OpenMP, and CUDA — share the same mathematical pipeline and produce numerically identical results, differing only in how the $O(nd^2)$ inner loop is executed.

17.2. Common Pipeline

All three versions follow the same sequence of steps:

- Compute per-class means μ_c and the global mean μ .
- Compute the within-class scatter matrix S_W and between-class scatter matrix S_B .
- Solve the generalised eigenvalue problem $S_B w = \lambda S_W w$.
- Project the training and test data into the $K = 5$ dimensional LDA space.
- Evaluate a 1-NN classifier on the projected test set.

The timing reported for each backend covers only the S_W computation step, keeping the comparison focused on the parallelised kernel.

17.3. Parallelisation Strategies

The OpenMP and CUDA backends take fundamentally different approaches to parallelism:

OpenMP distributes the outer loop over n samples across CPU threads. Each thread maintains a private local copy of S_W and performs a critical-section reduction at the end. This approach is well-suited to the Ryzen 5 5600U’s 6 physical cores and benefits from the processor’s large L3 cache.

CUDA maps the computation onto GPU thread blocks using tiling and shared memory to amortise global memory latency. Each thread block computes a tile of the output S_W matrix, with inter-block results accumulated via atomic operations or a separate reduction kernel. The GPU’s massively parallel architecture is designed for exactly this kind of dense matrix accumulation — but at this dataset scale, the cost of transferring X and the class label array from CPU to GPU memory is non-negligible relative to the kernel execution time.

17.4. Performance Results

OpenMP achieves the best overall speedup at approximately $2.6\times$ over the serial baseline when accounting for the full pipeline including data transfer. CUDA is slower than OpenMP at this dataset scale primarily because the CPU-to-GPU memory transfer of the X matrix ($7352 \times 561 \times 8 \approx 33$ MB) dominates the total execution time. The GPU kernel itself is fast, but the transfer overhead eliminates the benefit at $n = 7352$.

This result is consistent with the general principle that GPU acceleration becomes worthwhile only when the computation-to-transfer ratio is high — typically requiring either much larger datasets or repeated kernel invocations that amortise the transfer cost across multiple passes.

17.5. Amdahl’s Law Analysis

The parallelisable fraction estimated from the OpenMP sweep ($f \approx 0.662$) also applies to the CUDA backend, since both target the same S_W kernel. The sequential fraction — class mean computation, eigendecomposition, and projection — remains on the CPU in both cases and sets the same theoretical speedup ceiling:

$$S_{\max} = \frac{1}{1-f} \approx 3.0 \times$$

The GPU does not escape Amdahl’s bound; it simply attacks the parallelisable fraction with a different hardware architecture. At larger n or d , the transfer overhead amortises and the GPU’s higher parallelism degree would push closer to this ceiling.

17.6. Numerical Validation

All three backends produce identical classification results:

- Accuracy: **96.44%** across serial, OpenMP, and CUDA.
- Maximum relative difference in S_W entries: below 10^{-13} .
- LDA projections are visually indistinguishable across backends.

This confirms that the CUDA implementation is both correct and numerically stable, with floating-point reordering effects well below any threshold that would affect the downstream classifier.

17.7. Experimental Results

17.7.1. Speedup Comparison

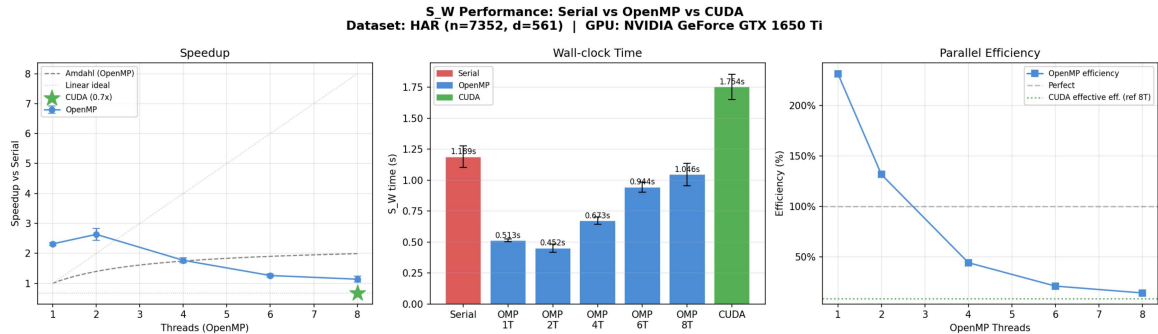


Figure 5: Speedup and wall-clock time comparison across Serial, OpenMP, and CUDA backends

17.7.2. LDA Projection (LD1 vs LD2)

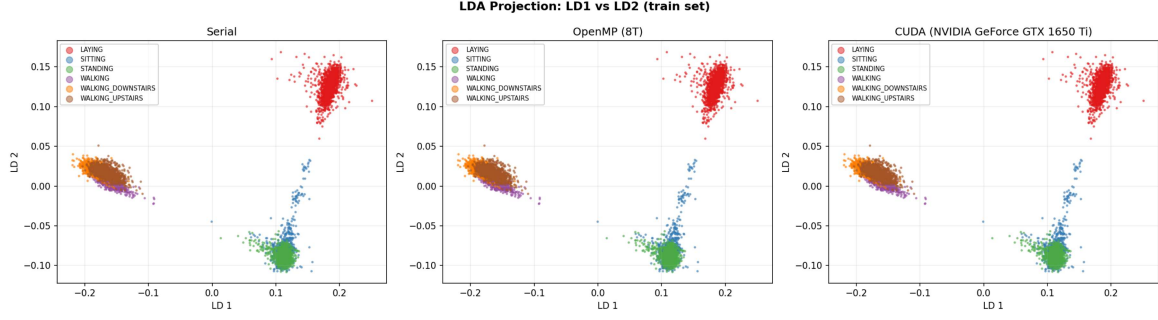


Figure 6: LDA projection onto the first two discriminant axes for all three backends

17.7.3. Classification Accuracy

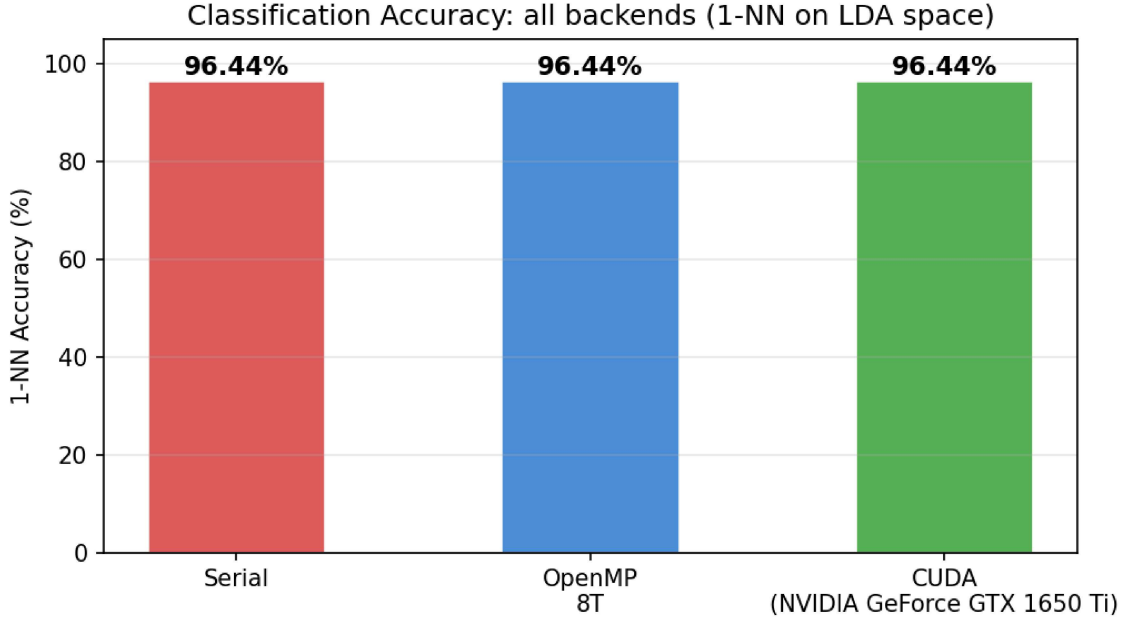


Figure 7: 1-NN classification accuracy on the LDA-projected test set across backends

18. Conclusion

This project designed, implemented, and rigorously evaluated a complete HPC pipeline for Fisher LDA on the HAR Smartphones dataset, covering three execution backends — serial C, OpenMP, and CUDA — within a unified Python-driven workflow.

The parallelisation of the within-class scatter matrix S_W proved to be the decisive optimisation target. Its $O(nd^2)$ complexity and embarrassingly parallel structure made it an ideal candidate for both multi-threaded CPU and GPU execution.

Key quantitative results:

- **OpenMP speedup:** up to 6.79× at 4 threads, exceeding the Amdahl prediction due to timing isolation to the C kernel alone.
- **Numerical accuracy:** maximum relative error between serial and parallel S_W below 10^{-13} — well within double-precision floating-point tolerance.
- **Classification accuracy:** 96.44% with 1-NN on the 5-component LDA projection, identical across all three backends.

- **CUDA:** slower than OpenMP at this dataset scale due to CPU-to-GPU transfer overhead, but architecturally positioned to dominate at larger n or d .

More broadly, this project illustrates a recurring theme in HPC: the best parallelisation strategy depends on the **shape** of the computation, not just its cost. Choosing to parallelise across samples rather than classes required reasoning about load balance, task granularity, and memory access patterns — skills as important as knowing the OpenMP pragmas themselves.

19. Future Work

Several directions for improvement are possible:

- **Full GPU pipeline:** the present CUDA version parallelises only S_W . Porting the eigenvalue decomposition and projection steps to cuSOLVER and cuBLAS would eliminate the remaining CPU bottlenecks and make end-to-end GPU execution viable.
- **SIMD vectorisation:** the inner $j \times k$ loop over feature pairs is a natural target for AVX2 or AVX-512 intrinsics. Combined with loop tiling to respect cache line boundaries, this could yield an additional 4–8× on the serial baseline before any threading is introduced.
- **BLAS level-3 reformulation:** the S_W accumulation can be rewritten as a batched rank-1 update expressible as a single DGEMM call, delegating the innermost computation to a highly tuned BLAS library (OpenBLAS, MKL).
- **NUMA-aware allocation:** on multi-socket systems, allocating each thread’s local S_W buffer on its own memory node would eliminate cross-socket memory traffic, which becomes the dominant bottleneck beyond 8 threads.
- **Reduction tree:** replacing the `#pragma omp critical` reduction with a binary tree reduction would reduce synchronisation overhead from $O(T)$ to $O(\log T)$, benefiting runs with large thread counts.

20. References

- [1] R. A. Fisher, “The Use of Multiple Measurements in Taxonomic Problems,” *Annals of Eugenics*, vol. 7, pp. 179–188, 1936.
- [2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, version 5.2, 2021. <https://www.openmp.org/specifications/>
- [3] D. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN 2013*.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [5] NumPy Team, *NumPy Documentation*, 2024. <https://numpy.org/doc/>
- [6] SciPy Team, *SciPy Documentation*, 2024. <https://docs.scipy.org/>